

Homework



Homework

- In today's homework, you will continue with the Restful API app from the demo exercise.
- This is what we have completed so far (in green): **C R U D**
- Here are the steps for this part 2 of the app:
 1. Add a **Create** button to post new item
 2. Add an **Update** button to put an existing item
 3. Add a **Delete** button to delete an existing item
- How they work:
 - All the above API calls will require their respective handlers and components.
 - The handler will pass data to the component via the calling of JSX element.
 - The component will output new data to the view.



Homework

Task 1: Add a Create button

- This button will be created inside the component. Start by creating a new file in the src folder – **CreateList.js**. Add the following scripts:

```
import React from "react";

function CreateList(props) {
  return (
    <React.Fragment>
    </React.Fragment>
  );
}

export default CreateList;
```

- Before we add the button, let's go over our strategy of how this button works and what happens after that.
- The button will opens a lightbox (Modal) that contains a simple form with two input text boxes.
- We will use react-bootstrap Modal to help us on this. You can find an example code here:

<https://react-bootstrap.github.io/components/modal/>

- And go ahead and install react-bootstrap:

npm install react-bootstrap



Homework

- Next we'll import the Modal and Button components from the library:

```
import React from "react";  
import { Modal, Button } from "react-bootstrap";
```

- The functionality of the Modal uses React's state to control the show and hide of the modal. The most updated method of course is using useState. But we need to import this before we can use this function:

```
import React, { useState } from "react";  
import { Modal, Button } from "react-bootstrap";
```

- Now we'll add the useState() function to handle the show/hide methods:

```
function CreateList(props) {  
  const [show, setShow] = useState(false);  
  
  const handleClose = () => setShow(false);  
  const handleShow = () => setShow(true);  
  return (  

```



Homework

- Next we'll add the button:

```
<React.Fragment>
  <Button variant="primary" onClick={handleShow}>
    Create New List
  </Button>
</React.Fragment>
```

- Below this button, we'll add the modal which comes with a header, body and footer:

```
<Modal show={show} onHide={handleClose}>
  <Modal.Header closeButton>
    <Modal.Title>New List</Modal.Title>
  </Modal.Header>
  <Modal.Body>

  </Modal.Body>
  <Modal.Footer>
    <Button variant="secondary" onClick={handleClose}>
      Close
    </Button>
    <Button variant="primary" onClick={handleClose}>
      Create
    </Button>
  </Modal.Footer>
</Modal>
</React.Fragment>
```



Homework

- Within the body, we'll create the two input text boxes:

```
<Modal.Body>
  <input
    type="text"
    placeholder="Title"
    name="title"
    value={props.singledata.title}
    onChange={props.handleChange}
    className="d-block my-3"
  />
  <input
    type="text"
    placeholder="Author"
    name="author"
    value={props.singledata.author}
    onChange={props.handleChange}
    className="d-block my-3"
  />
</Modal.Body>
```



Homework

- Our CreateList component is done. Let's move on to App.js. First import the CreateList component:

```
import Lists from "../Lists";  
import CreateList from "../CreateList";
```

- Then within the return and below the button, call this CreateList component while passing data from the state to it (this data is already set in the input's value attribute):

```
    </button>  
    <CreateList singledata={this.state.singledata} />  
  </span>  
  {listTable}
```



Homework

- Let's test this out: **npm run dev** , and this is what we have so far:



Get Lists	Create New List	
#	Title	Author

Get Lists	Create New List	
#	Title	

New List

×Close

Close

Create



Homework

- The create form don't work at this time, but we will do so now. So here's the plan:
 - As user enter information in the text box (onChange), the entered character will be captured and stored in the state. The value attribute in the box will be transmit the data entered to a handler that in turn will store in the state (setState). This handler will handle any of the API calls ie. post, put and delete.
 - To post/put/delete this data on the JSON database, we will need separate handlers to do just that.

- Our first task is to set up the handler that handles any incoming API calls. We will create this handler between the getList() method and the render():

```
handleChange = (event) => {  
  let title = this.state.singledata.title;  
  let author = this.state.singledata.author;  
  if (event.target.name === "title") title = event.target.value;  
  else author = event.target.value;  
  
  this.setState({  
    singledata: {  
      title: title,  
      author: author  
    }  
  });  
}
```

```
render() {
```



Homework

- Let's add the call of this API call handler in the CreateList calling in the return:

```
</button>
<CreateList
  singledata={this.state.singledata}
  handleChange={this.handleChange}
/>
```

- This handler is already set in the input text boxes in CreateList using the onChange event.
- The second task is to create the handler to handle the data posting to JSON server. We will create this handler below the previous handler handleChange().



Homework

- Just like the getList handler, we will use the Fetch API to connect to the array object on the server. We will use the POST method for this, along with a header and body.
- Within the body, we will use the JSON.stringify() method to convert the data to JSON string. (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/stringify).
- Using the Fetch API promises (then) to update (setState) the properties on the database file.

```
createList = () => {  
  fetch("http://localhost:5000/posts", {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json"  
    },  
    body: JSON.stringify(this.state.singledata)  
  }).then(  
    this.setState({  
      singledata: {  
        title: "",  
        author: ""  
      }  
    })  
  );  
}  
  
render() {
```



Homework

- Now, to make this create button to function, we need to add this post handler in the CreateList component calling:

```
</button>
<CreateList
  singledata={this.state.singledata}
  handleChange={this.handleChange}
  createList={this.createList}
/>
```

- And lastly, on the create button in CreateList component, we will add that to the onClick event calling of this handler through the attribute that's passed to it as props. In the create button within the modal footer, convert the onClick event to a callback arrow function

```
Button
  variant="primary"
  onClick={() => {
    handleClose();
    props.createList();
  }}
>
  Create
</Button>
```



Homework

- Time to test drive the app again...

Get Lists

Create New List

#	Title
1	Lord of The Rings
2	The Alchemist
3	Da Vinci Code
4	A Tale of Two Cities

New List

×Close

Nineteen Eighty-Four

George Orwell

Close

Create

Get Lists

Create New List

#	Title	Author
1	Lord of The Rings	J.R.R. Tolkien
2	The Alchemist	Paul Coelho
3	Da Vinci Code	Dan Brown
4	A Tale of Two Cities	Charles Dickens
5	Nineteen Eighty-Four	George Orwell



Homework

- Now that you saw how items are added on the view, how does that look on the server? If you recall, the POST method in the handler is suppose to post this new data in the database file.
- Let's check it out. Open db.json. The new data is added to the file. Sweet!

```
    "id": 4,  
    "title": "A Tale of Two Cities",  
    "author": "Charles Dickens"  
  },  
  {  
    "title": "Nineteen Eighty-Four",  
    "author": "George Orwell",  
    "id": 5  
  }  
]  
}
```



Homework

Task 2: Add an Update button

- This button will appear on every item on the list. This is because clicking on the update button only updates that item, not the entire list.
- Start by creating a new file in the src folder and save it as **UpdateList.js**. Insert these scripts:

```
import React, { useState } from "react";
import { Modal, Button } from "react-bootstrap";

function UpdateList(props) {
  return (
    <React.Fragment>

    </React.Fragment>
  )
}

export default UpdateList;
```



Homework

- Just like the Create button, the Update button will also launch a modal that contains a form. So therefore, we'll set up the modal next along with useState():

```
function UpdateList(props) {  
  const [show, setShow] = useState(false);  
  
  const handleClose = () => setShow(false);  
  const handleShow = () => setShow(true);  
  return (  
    <React.Fragment>
```

- Add the Update button within React.Fragment:

```
<React.Fragment>  
  <Button variant="primary" onClick={handleShow}>  
    Update  
  </Button>  
</React.Fragment>
```



Homework

- Below the button (still within the React.Fragment), add the modal:

```
</Button>  
<Modal show={show} onHide={handleClose}>  
  <Modal.Header closeButton>  
    <Modal.Title>Update List</Modal.Title>  
  </Modal.Header>  
  <Modal.Body></Modal.Body>  
  <Modal.Footer></Modal.Footer>  
</Modal>
```

- Now add the two same input text boxes within the body:

```
<Modal.Body>  
  <input type="text"  
    placeholder="Title"  
    name="title"  
    value={props.singledata.title}  
    onChange={props.handleChange}  
    className="d-block my-3"  
  >  
</input>  
  <input type="text"  
    placeholder="Author"  
    name="author"  
    value={props.singledata.author}  
    onChange={props.handleChange}  
    className="d-block my-3"  
  >  
</input>  
</Modal.Body>
```



Homework

- Within the footer, add the same two buttons:
- Now let's get the Update button (at the top before the modal) to work. This button will first get the list item id that is being clicked on. This id plus the event type will be passed to a handler to get the item (single data) from the database so it can be ready for any updates.

```
<Modal.Footer>
  <Button variant="secondary" onClick={handleClose}>
    Close
  </Button>
  <Button variant="primary" onClick={handleClose}>
    Update
  </Button>
</Modal.Footer>
```

```
<React.Fragment>
  <Button
    variant="primary"
    onClick={(evt) => {
      handleShow();
      props.getList(evt, props.elementId);
    }}
  >
    Update
  </Button>
```



Homework

- So let's head to App.js to create this new handler/method.
- This method will first have two parameters to accept data coming from the Update button (event and id).
- The method will first set the two properties to a string Loading...
- Then, connects to the array object in the database, specifically looking for the item (id) in question.
- If found, set the two data into the state for output in the modal.

```
getList = (event, id) => {  
  this.setState(  
    {  
      singledata: {  
        title: "Loading...",  
        author: "Loading..."  
      }  
    },  
    () => {  
      fetch("http://localhost:5000/posts/" + id)  
        .then(res => res.json())  
        .then(result => {  
          this.setState({  
            singledata: {  
              title: result.title,  
              author: result.author ? result.author : ""  
            }  
          });  
        });  
    })  
  );  
}
```



Homework

- Now on to **Lists.js**. Why? Because that's where the Update button will be. But first import **UpdateList**:
- We will now call the UpdateList component in a new column. In it we need to pass the item's id, the getList method, the yet-to-exist updateList method, the singledata and handleChange methods as well:

```
import React from 'react'  
import UpdateList from './UpdateList';  
import "bootstrap/dist/css/bootstrap.min.css";
```

```
<td>{element.author}</td>  
<td>  
  <UpdateList  
    elementId={element.id}  
    singledata={props.singledata}  
    getList={props.getList}  
    updateList={props.updateList}  
    handleChange={props.handleChange}  
  >  
  </UpdateList>  
</td>
```



Homework

- On App.js, we now need to pass data and handlers to the Lists component calling:

```
<Lists
  alldata={this.state.alldata}
  singledata={this.state.singledata}
  getList={this.getList}
  updateList={this.updateList}
  handleChange={this.handleChange}
/>
```

- Now create the updateList method using PUT method. Place it between the last method and render():

```
updateList= (event, id) => {
  fetch("http://localhost:5000/posts/" + id, {
    method: "PUT",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(this.state.singledata)
  })
  .then(res => res.json())
  .then(result => {
    this.setState({
      singledata: {
        title: "",
        author: ""
      }
    });
  });
}

render() {
```



Homework

- On this `updateList` method, notice two receiving data parameters – `event` and `id`. These will be passed from the Update button from the `UpdateLists` component, which we will head there now to add that event. Because the event will call multiple methods, it will be converted to a callback arrow function:

```
<Button
  variant="primary"
  onClick={(event) => {
    handleClose();
    props.updateList(event, props.elementId);
  }}
>
  Update
</Button>
</Modal.Footer>
```

Note:

The event and `elementId` will be passed to the `updateList` method as arguments.



Homework

- The last step is to update the entire list that is displayed in the view once the update is completed on the database. We do this by calling the `getLists` method and this should go at the end in the **updateList** method:

```
        author: ""  
      }  
    });  
    this.getLists();  
  });  
}  
  
render() {
```



Homework

- It's time to give it a test. We'll test the last update button.

Get Lists		Create New List		
#	Title	Author	Update	Delete
1	Lord of The Rings	J.R.R. Tolkien	<button>Update</button>	
2	The Alchemist	Paul Coelho	<button>Update</button>	
3	Da Vinci Code	Dan Brown	<button>Update</button>	
4	A Tale of Two Cities	Charles Dickens	<button>Update</button>	
5	Nineteen Eighty-Four	George Orwell	<button>Update</button>	





Homework

- Modal opens. Click Update button after the change.

Remove the dash

The screenshot shows a web application interface. On the left, there is a table with 5 rows of book data. On the right, an 'Update List' modal is open. The modal contains two input fields: the first contains 'Nineteen Eighty Four' and the second contains 'George Orwell'. A red arrow points from the text 'Remove the dash' to the 'Four' in the first input field. The modal has a 'Close' button and an 'Update' button. The background table has columns '#', 'Title', and an 'Update' button for each row.

#	Title
1	Lord of The Rings
2	The Alchemist
3	Da Vinci Code
4	A Tale of Two Cities
5	Nineteen Eighty-Four

Update List

Close Update



Homework

- Modal closes. Notice the lists is reloaded with the new update.

Get Lists Create New List			
#	Title	Author	Update
1	Lord of The Rings	J.R.R. Tolkien	Update
2	The Alchemist	Paul Coelho	Update
3	Da Vinci Code	Dan Brown	Update
4	A Tale of Two Cities	Charles Dickens	Update
5	Nineteen Eighty Four	George Orwell	Update



Homework

Task 3: Add a Delete button

- The Delete button will also be displayed on the lists, next to the Update button.
- The steps for this task is quite similar to the update. First, let's create the delete list component. Create a new file in the src folder and save as **DeleteList.js**. In it, start with the following setup:

```
import React, { useState } from "react";
import { Modal, Button } from "react-bootstrap";

function DeleteList(props) {
  return (
    <React.Fragment>

    </React.Fragment>
  )
}

export default DeleteList;
```



Homework

- Next, add useState and methods to set the show/hide of the modal:

```
function DeleteList(props) {  
  const [show, setShow] = useState(false);  
  const handleClose = () => setShow(false);  
  const handleShow = () => setShow(true);  
  return (  
    <React.Fragment>  
  
    </React.Fragment>  
  )  
}
```

- Add the Delete button with the exact same attributes as the Update button:

```
<React.Fragment>  
  <Button  
    variant="primary"  
    onClick={(evt) => {  
      handleShow();  
      props.getList(evt, props.elementId);  
    }}  
  >  
    Delete  
  </Button>  
</React.Fragment>
```



Homework

- We'll add the modal below the button:

```
</Button>
<Modal show={show} onHide={handleClose}>
  <Modal.Header closeButton>
    <Modal.Title>Delete List</Modal.Title>
  </Modal.Header>
  <Modal.Body>

  </Modal.Body>
  <Modal.Footer></Modal.Footer>
</Modal>
</React.Fragment>
```

- Within the body, add the two input text box. Also an attribute disabled is added to disallow text entered in the boxes:

```
<Modal.Body>
  <input type="text"
    placeholder="Title"
    name="title"
    value={props.singledata.title}
    disabled={true}
    className="d-block my-3"
  >
</input>
  <input type="text"
    placeholder="Author"
    name="author"
    value={props.singledata.author}
    disabled={true}
    className="d-block my-3"
  >
</input>
</Modal.Body>
```



Homework

- And in the footer, we'll add similar buttons with the same attributes, except passing data to the deleteList handler:

```
<Modal.Footer>
  <Button variant="secondary" onClick={handleClose}>
    Close
  </Button>
  <Button
    variant="primary"
    onClick={(event) => {
      handleClose();
      props.deleteList(event, props.elementId);
    }}
  >
    Delete
  </Button>
</Modal.Footer>
```

- Moving on to **Lists.js**. Just like UpdateList, we will call the DeleteList component here because this button will appear next to the Update button. But first let's import the component:

```
import React from 'react'
import UpdateList from './UpdateList';
import DeleteList from './DeleteList';
import "bootstrap/dist/css/bootstrap.min.css";
```



Homework

- We'll call the DeleteList component in the next column. It will pass similar data and handlers like the UpdateList component, except the deleteList handler:

```
</td>
<td>
  <DeleteList
    elementId={element.id}
    singledata={props.singledata}
    getList={props.getList}
    deleteList={props.deleteList}
  >
  </DeleteList>
</td>
</tr>
```

- On **App.js**, we'll in turn add the deleteList handler to be passed to the Lists component.

```
<Lists
  alldata={this.state.alldata}
  singledata={this.state.singledata}
  getList={this.getList}
  updateList={this.updateList}
  deleteList={this.deleteList}
  handleChange={this.handleChange}
/>
```



Homework

- Next, we will create the deleteList method in App.js, using the DELETE method. This method will go above the render().
- It contains the same two parameters (event and id). They will receive data passed from the Delete button in the DeleteList component via Lists component.
- The getLists is to refresh the lists once the item in the database is removed.

```
deleteList = (event, id) => {  
  fetch("http://localhost:5000/posts/" + id, {  
    method: "DELETE"  
  })  
    .then(res => res.json())  
    .then(result => {  
      this.setState({  
        singledata: {  
          title: "",  
          author: ""  
        }  
      });  
      this.getLists();  
    });  
}
```

```
render() {
```



Homework

- It's time to take it for a test again. We'll delete the last item.

Get Lists		Create New List		
#	Title	Author	Update	Delete
1	Lord of The Rings	J.R.R. Tolkien	Update	Delete
2	The Alchemist	Paul Coelho	Update	Delete
3	Da Vinci Code	Dan Brown	Update	Delete
4	A Tale of Two Cities	Charles Dickens	Update	Delete
5	Nineteen Eighty Four	George Orwell	Update	Delete





Homework

- This should open the modal. Notice the text boxes are disabled. Click the Delete button.

#	Title		Delete
1	Lord of The Rings		Delete
2	The Alchemist		Delete
3	Da Vinci Code		Delete
4	A Tale of Two Cities		Delete
5	Nineteen Eighty Four	George Orwell	Update Delete

Delete List

×Close

Nineteen Eighty Four

George Orwell

Close Delete



Homework

- The modal box closes. The lists is reloaded with the last item now removed.

Get Lists Create New List				
#	Title	Author	Update	Delete
1	Lord of The Rings	J.R.R. Tolkien	Update	Delete
2	The Alchemist	Paul Coelho	Update	Delete
3	Da Vinci Code	Dan Brown	Update	Delete
4	A Tale of Two Cities	Charles Dickens	Update	Delete



Homework

- Let's check out the db.json file.
- Notice the last item (#5) has been deleted in the array object.

```
{  
  "id": 4,  
  "title": "A Tale of Two Cities",  
  "author": "Charles Dickens"  
}
```



Homework

- **DUE:**

On or before this Sunday, 10.30PM PT.

- **Submission:**

Post GitHub URL (second commit) on GAP Week 3 Day 2 Homework



Connect with Us (WEB403/603/803)

Professor

Rich Loke

richloke@westcliff.edu

Teaching
Assistant

Elizabeth Kipp

elizabethkipp@westcliff.edu

Teaching
Assistant

Erin Tustin

erintustin@westcliff.edu

End of Presentation